AiQ Assessment Architecture

Version 2.2 — Engine Remediation Work Package

Document status: Released
Date: 23 April 2026
Platform version: 059d15b8
Preceding version: v2.1 (checkpoint ea81412f, 21 April 2026)

Table of Contents

---

1. Purpose and Scope

AiQ is an enterprise HR capability intelligence platform that measures how HR professionals actually use AI in their work. The assessment engine is the core of the platform: it administers adaptive scenario-based assessments, scores responses against a calibrated signal system, detects gaming behaviour, and produces a readiness classification with a plain-language explanation.

This document describes the architecture of the assessment engine as of version 2.2. It covers all server-side engine modules, the tRPC procedure layer, the scoring and classification pipeline, and the test suite. It is intended for engineers extending the platform, data scientists recalibrating the scoring model, and technical reviewers conducting due diligence.

The v2.2 release is a targeted remediation work package addressing four categories of defect identified in the v2.1 post-launch review:

- **WS1** — Scoring engine correctness: the v2.1 mean-based formula violated monotonicity and diluted scores for high-evidence sessions.
- **WS2** — Anti-gaming engine recalibration: the outcome-conditional pattern detector was not yet implemented; role-aware thresholds were not applied.
- **WS3** — LLM item quality gate: the gate existed but lacked a PII sanitiser, vendor allow-list, and bias detector.
- **WS4** — Participant experience: the save-and-resume feature lacked a formal 48-hour window and model version pinning.

---

2. Engine Inventory

The assessment engine comprises 11 TypeScript modules in `server/assessment/` and one tRPC router in `server/routers/assessment.ts`. The table below summarises each module, its line count, and its primary responsibility.

Module	Lines	Responsibility
`adaptiveEngine.ts`	1,090	LLM scenario generation, role archetypes, item sequencing
`scoringEngine.ts`	780	Signal scoring, capability scores, readiness classification
`antiGamingEngine.ts`	362	Gaming pattern detection, trap injection, scrutiny escalation
`sessionController.ts`	564	3-phase session lifecycle, minimum evidence model
`contradictionEngine.ts`	328	Inconsistency detection, follow-up probe injection
`normEngine.ts`	378	Normative percentile scoring against synthetic baselines
`roleArchetypes.ts`	322	22 HR role archetypes, capability weightings, thresholds
`llmQualityGate.ts`	250	4-checker quality gate, circuit breaker, review queue
`classificationConfidenceGate.ts`	175	Confidence band thresholds, safe-classification gate
`scoringConfig.ts`	102	DB-backed scoring config loader with 5-minute cache
`featureFlags.ts`	75	Feature flag helpers (env-var backed)
`routers/assessment.ts`	1,992	tRPC procedures: session lifecycle, scoring, results

The total engine codebase is approximately 6,400 lines of TypeScript, excluding tests.

---

3. Signal System

The signal system is the foundation of the scoring model. Each assessment item option carries a set of **signal deltas** — unsigned magnitudes (0.0–3.0) keyed to one of 14 signal dimensions. The sign of each contribution is determined by the option's **outcome class** multiplied by the `OUTCOME_SCORE_MODIFIER` table, not by the stored delta value. This prevents the double-sign problem that would arise if failure options stored negative deltas.

3.1 Signal Dimensions

The engine operates on exactly **22 canonical signal keys**, seeded in migration `0009_signal_key_constraints.sql` and enforced by the `canonical_signal_keys` reference table. The previous v2.2 document listed only 16 signals and included a phantom signal (`validation_bypass_risk`) that does not exist in the codebase. This section has been corrected in the Completion Pass to reflect the full 22-signal inventory.

Signal Key	Maps to Capability	Risk Signal
`execution_quality`	execution	No
`prioritisation_quality`	execution	No
`validation_accuracy`	execution	No
`timing_integrity`	execution	No
`consistency_index`	execution	No
`judgement_quality`	judgement	No
`discrimination_quality`	judgement	No
`over_caution_risk`	judgement	**Yes**
`avoidance_risk`	judgement	**Yes**
`calibration_index`	judgement	No
`governance_quality`	governance	No
`over_reliance_risk`	governance	**Yes**
`blind_acceptance_risk`	governance	**Yes**
`contradiction_index`	governance	**Yes**
`governance_bypass_risk`	governance	**Yes**
`hallucination_acceptance_risk`	governance	**Yes**
`appropriateness_boundary`	appropriateness	No
`automation_expansion_risk`	appropriateness	**Yes**
`cosmetic_focus_risk`	appropriateness	**Yes**
`unsafe_hr_decision_risk`	appropriateness	**Yes**
`workflow_application_quality`	workflow	No
`data_interpretation_quality`	data_interpretation	No

Risk signals receive an additional multiplier from `RISK_MULTIPLIERS` when the item's `riskLevel` is `High` or `Critical`. This ensures that governance failures on high-stakes items carry proportionally greater weight.

3.2 Outcome Class Modifiers

The `OUTCOME_SCORE_MODIFIER` table defines the sign and magnitude of each outcome class's contribution to the signal score:

Outcome Class	Modifier	Effect
`strong`	+1.0	Full positive contribution
`acceptable`	+0.6	Partial positive contribution
`weak`	-0.3	Small negative contribution
`failure`	-0.7	Significant negative contribution
`critical_failure`	-1.5	Large negative contribution (can exceed base delta)

3.3 Difficulty Weights

Items are weighted by difficulty level. Level 4 items (advanced, introduced in v2.1) carry the highest weight:

Difficulty	Weight
1 (introductory)	0.80
2 (standard)	1.00
3 (challenging)	1.20
4 (advanced)	1.35

---

4. Scoring Engine — v2.2 Changes

4.1 v2.1 Formula (Legacy)

The v2.1 formula computed capability scores using a mean-based approach:

```
avgDelta = sum(signalDeltas) / count
scaleFactor = f(count)   // shrinks as count grows
score = intercept + avgDelta × scaleFactor
```

This formula violated **monotonicity**: adding a positive-delta answer to a session with many existing answers could *lower* the score because the scale factor decreased faster than the new delta raised

the average. It also produced artificially low scores for high-evidence sessions (many answers).

4.2 v2.2 Formula (Sum+Clip)

The v2.2 formula replaces the mean-based approach with a sum-and-clip formula:

```
clipped = clip(sum(signalDeltas), -contributionCap, +contributionCap)
score   = clip(intercept + clipped × contributionMultiplier, 0, 100)
```

The default calibration parameters are:

Parameter	Default Value	Source
`intercept`	50	Scoring config DB row
`contributionCap`	8.0	Calibrated against anchor cases
`contributionMultiplier`	6.25	Calibrated against anchor cases

These defaults were calibrated against four synthetic anchor cases:

Anchor	Description	Expected Score
A	5 strong answers (delta=0.8, Medium, diff=2)	75
B	5 critical_failure answers (delta=0.8, Critical, diff=3)	0
C	3 strong + 4 failure answers (mixed)	49 (35–65 band)
D	Monotonicity: N+1 strong answers ≥ N strong answers	score(N+1) ≥ score(N)

Thin-Signal Capability Verification (Apr 23 2026)

The original anchor cases (A–D) were written exclusively against `governance_quality`, a capability with six contributing signals. A subsequent audit confirmed that the formula is signal-count-agnostic — it sums weighted deltas regardless of how many signals contribute to a capability — and therefore generalises correctly to single-signal capabilities without modification.

The two thin-signal capabilities were verified against all four anchor conditions:

Capability	Signals	Anchor A	Anchor B	Anchor C	Monotonicity
`workflow`	1 (`workflow_application_quality`)	75	0	49	✓
`data_interpretation`	1 (`data_interpretation_quality`)	75	0	49	✓
`governance` (reference)	6	75	0	49	✓

All three capabilities produce identical scores for equivalent inputs, confirming calibration parity. Additional thin-signal-specific cases were verified:

Case	Input	Score	Notes
Single strong answer	delta=0.8, Medium, diff=2	55	Above intercept — correct
Single critical_failure (small delta)	delta=0.8, Critical, diff=3	36	Does NOT floor to 0 — clip does not fire
Single critical_failure (large delta)	delta=5.0, Critical, diff=3	0	Clip fires at −8.0 — intentional
1 strong + 1 failure	delta=0.8, Medium, diff=2 each	51	Near intercept — correct

Cross-capability isolation was also verified: workflow answers do not affect `data_interpretation` scores and vice versa. All 19 thin-signal tests are in `server/scoring.thin-signal.test.ts`.

4.3 Backward Compatibility

The v2.1 formula is preserved in the codebase and activated automatically for sessions that were started under a v2.1 scoring config row. The `scoringConfigVersionAtStart` field on `assessmentSessions` records which config version was active when the session began, ensuring in-flight sessions complete under the formula they started with.

4.4 Scoring Config Loader

The `scoringConfig.ts` module loads the active scoring config from the database with a 5-minute in-memory cache. If the database is unavailable, it falls back to the `DEFAULT_CONFIG` (v2.1 parameters). The cache can be invalidated programmatically after an admin updates the active config.

---

5. Failure Mode Detection

The `detectFailureModes` function in `scoringEngine.ts` scans the answer sequence for patterns that indicate systematic governance failures. It returns a `FailureModeResult` with:

- `detected` — whether any failure mode was found
- `modes` — array of unique failure mode identifiers
- `governanceFlag` — whether any blocking governance failure mode was detected
- `classificationImpact` — `"none"`, `"downgrade"`, or `"block"`

5.1 Failure Mode Taxonomy

Mode	Trigger	Governance Flag
`blind_ai_acceptance`	`blind_acceptance_risk` signal delta < -1.5, or `BLIND_ACCEPT` event code	Yes
`governance_bypass`	`governance_bypass_risk` signal delta < -1.5, or `GOVERNANCE_BYPASS` event code	Yes
`hallucination_acceptance`	`hallucination_acceptance_risk` signal delta < -1.5	Yes
`unsafe_hr_decision`	`unsafe_hr_decision_risk` signal delta < -1.5	Yes
`critical_failure_response`	`critical_failure` outcome class	Yes
`poor_validation`	`validation_accuracy` signal delta < -2.0	No
`over_reliance`	`over_reliance_risk` signal delta < -1.0	No

5.2 Configurable Thresholds (WS1.2)

The minimum evidence thresholds for blocking and downgrading classifications are configurable via the scoring config:

Parameter	Default	Effect
`blockingFailureMinItems`	2	Minimum number of **answer-level** blocking failure mode triggers required to produce a `block` classification impact
`downgradeFailureMinItems`	1	Minimum number of **answer-level** failure mode triggers (any type) required to produce a `downgrade`
`baseFailureThresholdMagnitude`	1.50	Per-answer weighted delta magnitude at which a failure mode triggers (e.g. `blind_acceptance_risk < -1.50`)
`catastrophicMarginMultiplier`	1.50	Multiplier applied to `baseFailureThresholdMagnitude` to determine the catastrophic single-item threshold (default: 1.50 × 1.50 = 2.25)
`atRiskConfidenceFloor`	0.35	Below this composite confidence, an `at_risk` classification is downgraded to `insufficient_evidence`
`provisionalConfidenceThreshold`	0.40	Below this composite confidence, `unknown_insufficient_evidence` is returned regardless of score
`confidenceFloor`	0.50	Below this composite confidence (but above `provisionalConfidenceThreshold`), `isProvisional = true` is set on the result
`minimumSafeClassificationConfidence`	0.55	Below this composite confidence, a `safe` classification is downgraded to `at_risk`

WS1.2 Item 1 (Apr 2026): `baseFailureThresholdMagnitude`, `catastrophicMarginMultiplier`, `atRiskConfidenceFloor`, `provisionalConfidenceThreshold`, `confidenceFloor`, and `minimumSafeClassificationConfidence` were previously hard-coded compile-time constants. They are now stored in `scoring_config` (migration `0019`) and loaded via `getActiveScoringConfig()`. All defaults reproduce the former hard-coded behaviour exactly.

The engine counts **per-answer occurrences** of failure mode triggers, not unique mode identifiers. Ten answers all triggering `blind_ai_acceptance` produce a `blockCount` of 10, which exceeds the default threshold of 2 and results in a `block`. This is the correct behaviour: the two-item threshold exists to prevent a *single bad answer* from blocking an otherwise strong session, not to require failures across multiple categories. A participant who catastrophically fails one governance mode on ten consecutive items should be blocked.

This semantics was corrected in the Completion Pass (previously the implementation incorrectly used unique-mode counting).

---

6. Anti-Gaming Engine

The `antiGamingEngine.ts` module detects behavioural patterns that suggest a participant is gaming the assessment rather than responding authentically. It operates on the full answer sequence and returns a `GamingAnalysis` object.

6.1 Pattern Taxonomy

Pattern	Trigger Condition	Governance Impact
`always_safe_choice`	>75% acceptable outcomes AND <10% strong outcomes	Scrutiny escalation
`always_escalate`	>70% escalation choices	Scrutiny escalation
`always_cautious`	>60% cautious/over-cautious answers	Scrutiny escalation
`option_position_bias`	>70% of answers select option A or option D	Trap injection
`speed_gaming`	>40% of answers submitted in <4 seconds	Scrutiny escalation
`inconsistent_responses`	Contradiction engine detects >3 inconsistencies	Follow-up probe injection
`polished_shallow`	High free-text length but low signal delta	Scrutiny escalation
`pattern_cycling`	Alternating outcome classes in a detectable cycle	Trap injection
`outcome_cycling`	Semantic cycling through outcome classes	Trap injection
`outcome_conditional_safe`	>70% high-risk answers are acceptable AND >70% low-risk answers are strong	Governance trap injection
`seniority_inconsistent`	Senior user (seniority ≥ 3) with >50% weak answers	Scrutiny escalation

6.2 WS2.1 — Outcome-Conditional Detection

The `outcome_conditional_safe` pattern detects a specific gaming strategy where participants give safe/acceptable answers on high-risk items (to avoid governance failures) while demonstrating strong performance on low-risk items (to maintain a high overall score). This pattern is only evaluated when:

- The session has at least 8 answers total

- At least 3 answers are on high-risk items (`riskLevel` = `High` or `Critical`)
- At least 3 answers are on low-risk items (`riskLevel` = `Low`)

When detected, the engine injects a governance-domain trap item (`interactionType: "governance_decision"`) to probe whether the participant's governance responses are genuinely cautious or strategically safe.

This feature is controlled by the `ANTI_GAMING_OUTCOME_CONDITIONAL` environment variable (default: `true`).

6.3 WS2.2 — Role-Aware Thresholds

The `DEFAULT_GAMING_THRESHOLDS` object defines the default detection thresholds. Role-specific overrides are applied based on the session's role archetype:

Role	`alwaysSafeAcceptableRate`	`speedGamingRate`	`alwaysEscalateRate`
Default (generalist)	0.75	0.40	0.70
Specialist	0.70	0.45	0.70
Leader	0.70	0.40	0.55

The `seniority_inconsistent` pattern is suppressed for junior users (seniority tier 1) because weak performance is expected and not indicative of gaming.

6.4 WS2.3 — Audit Event Codes

Every detected pattern maps to a standardised audit event code in `GAMING_AUDIT_EVENT_CODES` . These codes are written to the session metadata and the audit log, enabling post-hoc analysis of gaming detection rates.

---

7. LLM Item Quality Gate

The `llmQualityGate.ts` module runs four checker passes on every LLM-generated item before it is served to a participant. Items that fail any checker are routed to the human review queue and a fallback item is served instead.

7.1 Checker Pipeline

Checker	What it validates
`structural_completeness`	Required fields present; 3–5 options; at least one `strong` and one `weak` / `failure` option; difficulty in range 1–4
`pii_sanitiser`	No email addresses, UK phone numbers, or National Insurance numbers in any text field
`vendor_allow_list`	No specific AI product names (ChatGPT, Claude, Copilot, Gemini, Bard, GPT-4, etc.)
`bias_detector`	No gendered pronouns (he/she/his/her), age-related language (elderly, young), or disability-related language (disabled, handicapped)

All four checkers run on every item. The gate result records which checkers failed, enabling targeted review.

7.2 Circuit Breaker

If the LLM generation service fails 3 consecutive times within a 60-second window, the circuit breaker opens and all subsequent items are routed directly to the human review queue without attempting LLM generation. The circuit breaker resets after a successful generation.

7.3 Human Review Queue

Failed items are inserted into the `llm_item_review_queue` table with:

- The item JSON
- The failure reason (concatenated checker names)
- The session ID
- Status `pending`

A back-office reviewer can approve, reject, or auto-approve items in the queue.

---

8. Session Lifecycle and Save-and-Resume

8.1 Three-Phase Structure

Every assessment session progresses through three phases:

1. **Baseline** — Items drawn from the static content library (`content_scenarios`), ensuring every participant answers the same foundational items. Minimum 10 items.

2. **Adaptive** — Items generated by the LLM adaptive engine, targeting the participant's weakest capability domains. Minimum 20 items.
3. **Validation** — A subset of baseline items re-administered to detect inconsistency and confirm the adaptive phase results. Minimum 10 items.

The `MINIMUM_EVIDENCE` constants define the evidence thresholds that must be met before a session can be completed:

Constant	Value	Meaning
`totalItems`	20	Minimum total items answered
`signalsPerCapability`	3	Minimum signal observations per capability domain
`distinctInteractionTypes`	5	Minimum distinct interaction types observed
`highRiskProportion`	0.25	Minimum proportion of high-risk items
`targetItems`	49	Target item count for evidence depth normalisation

8.2 WS4.4 — Save-and-Resume (48-Hour Window)

A participant may pause their session from item 10 onwards. The session state is persisted to the database. The `getResumeState` procedure returns:

- `isResumeWindowOpen` — `true` if the session was last active within the past 48 hours
- `resumeWindowExpiresAt` — the exact timestamp at which the resume window closes
- `progressPct` — percentage of target items answered (rounded to nearest integer)

Sessions that exceed the 48-hour window are marked as `expired` and cannot be resumed. Partial results are preserved in the back-office for admin review.

8.3 WS4.5 — Model Version Pinning

When a session is paused, the LLM model version in use is pinned to the session metadata as `pinnedModelVersion`. When the session is resumed, the adaptive engine uses the pinned version rather than the current default. This ensures that resumed sessions are not affected by model updates that occurred during the pause window.

The default pinned version is `"adaptive-v2"`. The resolved version is returned by `getResumeState` as `pinnedModelVersion`.

---

9. Classification and Confidence Gate

9.1 Readiness States

The `classifyReadiness` function maps the scoring pipeline output to one of five readiness states:

State	Label	Colour	Governance Action
`safe`	AI-Ready	#10B981 (teal)	None
`at_risk`	Developing	#F59E0B (amber)	Supervised use recommended
`unsafe`	Not Yet Ready	#EF4444 (red)	Development required
`unknown`	Insufficient Evidence	#6B7280 (grey)	Provisional
`unknown_insufficient_evidence`	Insufficient Evidence	#6B7280 (grey)	Provisional

The `safe` state requires all of the following:

- Overall score ≥ 75
- No capability domain below its role-specific minimum safe threshold
- Risk band `low` or `medium`
- No blocking failure modes

9.2 Confidence Gate

The `classificationConfidenceGate.ts` module prevents a `safe` classification from being issued when the composite confidence score is below the minimum threshold (default: 0.50). This addresses the CPO credibility concern that a session with 0.35 composite confidence should not produce an `AI-Ready` classification.

Confidence bands:

Band	Score Range	Effect
`high`	≥ 0.75	No gate applied
`moderate`	0.55–0.74	No gate applied; caveat added to result
`low`	0.40–0.54	Caveat added; `safe` classification gated
`insufficient`	< 0.40	`safe` classification blocked; result is provisional

9.3 WS4.2 — Classification Explanation

The `getClassificationExplanation` procedure returns a structured plain-language explanation of the classification, including:

- `state` and `label` — the readiness state and its human-readable label
- `isProvisional` — `true` for unknown states or confidence < 0.4
- `topStrengths` — the two highest-scoring capability domains
- `topGaps` — the two lowest-scoring capability domains
- `factors` — an ordered array of contributing factors (overall score, confidence, governance flag, failure modes), each with a direction (`positive` , `neutral` , or `negative`) and a plain-language detail string
- `confidenceBand` — `"high"` , `"medium"` , or `"low"`
- `scoringConfigVersion` — the version of the scoring config used to produce this result

---

10. Normative Scoring

The `normEngine.ts` module provides percentile ranks so every capability score is contextualised relative to a reference population. The reference distributions are synthetic bootstraps derived from the scoring model's expected range.

10.1 Norm Group Version

Every result is stamped with `normGroupVersion: "synthetic-v1"` . When real completion data becomes available, the distributions will be recalibrated and the version incremented. Results from different norm versions are not directly comparable.

10.2 Distribution Parameters

Distributions are parameterised per role family × seniority tier × capability domain. Each distribution is a normal approximation (mean, stdDev) from which the percentile is computed analytically using the error function.

Role families: `generalist` , `specialist` , `analytics` , `leadership` , `coordinator` .
Seniority tiers: `junior` , `mid` , `senior` .

---

11. Adaptive Intelligence Layer (AIL)

The AIL processes completed assessment sessions to produce a persistent user intelligence profile. It runs asynchronously after session completion and does not block the scoring pipeline.

11.1 AIL Pipeline

1. **Persona classification** — classifies the participant into one of five personas based on their response patterns:
    - `strong_validator` — high governance quality, high validation accuracy
    - `overconfident_decision_maker` — high execution quality, low validation accuracy
    - `risk_averse_escalator` — high over-caution risk, low execution quality
    - `passive_deferrer` — high avoidance risk, low judgement quality
    - `governance_anchor_under_pressure` — high governance quality under high-risk conditions
2. **Difficulty adaptation** — adjusts the starting difficulty for future sessions based on the persona profile (gated by `PERSONA_ADAPTATION_ENABLED` flag).
3. **Learning plan generation** — generates a capability-mapped learning plan targeting the participant's weakest three capability domains.

11.2 Persona Adaptation Feature Flag

The `isPersonaAdaptationEnabled()` function reads the `PERSONA_ADAPTATION_ENABLED` environment variable. It defaults to `false`. The feature should only be enabled when the persona classification engine has been validated against at least 100 sessions per persona.

---

12. Feature Flags

All feature flags are backed by environment variables and evaluated at runtime. They are captured in session metadata at session start.

Flag	Env Var	Default	Description
Persona adaptation	`PERSONA_ADAPTATION_ENABLED`	`false`	Enable AIL persona-based difficulty adjustment
Outcome-conditional anti-gaming	`ANTI_GAMING_OUTCOME_CONDITIONAL`	`true`	Enable outcome-conditional gaming pattern detection
Validation phase randomisation	`VALIDATION_PHASE_ORDER_RANDOMISED`	`false`	Interleave validation items with adaptive items

---

13. Test Coverage

The v2.2 release adds 105 new tests across 6 new test suites, bringing the total to 237 tests across 13 test files.

Test File	Tests	Work Stream	Coverage
`scoring.v2-2.test.ts`	8	WS1.1	Sum+clip formula anchor cases A/B/C/D, monotonicity, boundary clamping, legacy fallback, output completeness
`failure-modes.v2-2.test.ts`	13	WS1.2	Default thresholds, configurable thresholds, event code triggers, non-blocking modes
`classification-explanation.test.ts`	22	WS4.2	isProvisional, confidence bands, topStrengths/topGaps, buildFactors
`anti-gaming.outcome-conditional.test.ts`	16	WS2.½.2/2.3	Outcome-conditional detection, feature flag, role-aware thresholds, audit event codes
`llm-checkers.test.ts`	27	WS3	All 4 checkers, circuit breaker, buildReviewQueueRow
`save-resume.test.ts`	19	WS4.4/4.5	48h window, model version pinning, progress computation, MINIMUM_EVIDENCE constants
`aiq.test.ts`	21	Core	Signal scoring, capability scores, readiness classification
`assessment.stress.test.ts`	44	Core	Session lifecycle, evidence model, role-aware thresholds
`ail.test.ts`	29	AIL	Adaptive difficulty engine, persona classification
`content.test.ts`	28	Content	Scenario library, option integrity
`debug.test.ts`	5	Session	Role-aware evidence thresholds
`waitlist.test.ts`	4	Auth	Waitlist flow
`auth.logout.test.ts`	1	Auth	Logout procedure

All 237 tests pass. TypeScript compilation produces 0 errors.

---

14. Known Gaps and Roadmap

The following items were identified in the v2.2 work package. Items marked **Resolved** were closed in the v2.2 completion pass (checkpoint 95372a68+).

Item	Work Stream	Priority	Status
Contribution breakdown JSON on `assessment_answers`	WS1.3	High	**Resolved**
Back-office contribution breakdown view	WS1.3	Medium	**Resolved**
Anti-gaming thresholds DB table (16 rows, 4 patterns × 4 tiers)	WS2.2	High	**Resolved**
Telemetry table (`assessment_answer_telemetry`) WS5.1 columns	WS5.1	Medium	**Resolved**
Session metadata WS5.2 columns	WS5.2	Medium	**Resolved**
Persona label softening (WS4.1)	WS4.1	High	**Resolved**
Back-office anti-gaming thresholds UI	WS2.2	Medium	**Resolved**
Back-office LLM review queue UI	WS3	Medium	**Resolved**
Back-office session review flags UI	WS4.3	Medium	**Resolved**
Feature flags: LLM_CHECKER_ENABLED, SAVE_AND_RESUME_ENABLED, PERSONA_LABEL_SOFTENING_ENABLED	Cross-cutting	Medium	**Resolved**
Required reasoning on high-signal items	S4	High	Deferred to v2.3
Governing constraint on all classifications	S3	Medium	Deferred to v2.3
Structured role picker (replaces free-text role hint)	S7	Medium	Deferred to v2.3
Organisation-configurable thresholds	S10	Medium	Deferred to v2.3
Content library expansion (22 new items)	S11	Low	Deferred to v2.3

---

15. Data Model Summary

The assessment engine reads from and writes to the following database tables:

Table	Purpose
`assessment_sessions`	Session lifecycle, phase, state, metadata
`assessment_answers`	Per-answer records with signal deltas, outcome class, timing
`assessment_scores`	Final scores, capability breakdown JSON, readiness state
`assessment_review_flags`	Participant-submitted flags for review
`assessment_answer_telemetry`	Behavioural telemetry (focus loss, scroll depth, revisions)
`llm_item_review_queue`	Items that failed the LLM quality gate
`scoring_config`	Active scoring config (intercept, multiplier, v2.2 params)
`content_scenarios`	Static baseline item bank (79 scenarios)
`content_scenario_options`	Options for each scenario (316 options)
`ail_user_profiles`	AIL persona profiles and intelligence history

---

16. Deployment and Configuration

The assessment engine runs as part of the Express/tRPC server. No separate deployment is required. The following environment variables control engine behaviour:

Variable	Required	Default	Description
`DATABASE_URL`	Yes	—	MySQL/TiDB connection string
`BUILT_IN_FORGE_API_KEY`	Yes	—	LLM API bearer token
`BUILT_IN_FORGE_API_URL`	Yes	—	LLM API base URL
`PERSONA_ADAPTATION_ENABLED`	No	`false`	Enable AIL persona adaptation
`ANTI_GAMING_OUTCOME_CONDITIONAL`	No	`true`	Enable outcome-conditional anti-gaming
`VALIDATION_PHASE_ORDER_RANDOMISED`	No	`false`	Randomise validation phase order
`LLM_CHECKER_ENABLED`	No	`true`	Enable LLM item quality gate (disable to bypass in CI/dev)
`SAVE_AND_RESUME_ENABLED`	No	`true`	Enable save-and-resume feature
`PERSONA_LABEL_SOFTENING_ENABLED`	No	`true`	Use softened participant-facing persona labels (WS4.1)

17. Security and Governance

All assessment procedures are protected by `protectedProcedure`, which requires a valid session cookie. The `getClassificationExplanation` procedure additionally verifies that the requesting user is the session owner before returning results.

The LLM quality gate PII sanitiser strips email addresses, phone numbers, and National Insurance numbers from all item text fields before they are served to participants. This prevents accidental PII leakage from LLM-generated content.

Gaming detection results and failure mode flags are written to the audit log and are accessible to tenant admins and super-admins only.

---

18. Performance Characteristics

The scoring pipeline is synchronous and CPU-bound. A full scoring run (signal computation, capability scores, failure mode detection, confidence profile, readiness classification) completes in under 5ms for a 49-item session on a single core.

LLM item generation is the primary latency source. The adaptive engine pre-generates the next item asynchronously after each answer submission, so the participant does not wait for generation during normal flow. Circuit breaker activation (3 consecutive LLM failures) routes to the static fallback bank and adds no latency.

The scoring config cache (5-minute TTL) prevents a DB round-trip on every score computation. Cache invalidation is synchronous and completes in under 1ms.

---

19. Accessibility and Internationalisation

All assessment UI components target WCAG 2.1 AA compliance. The assessment session page uses semantic HTML, visible focus rings, and ARIA labels on all interactive elements. The confidence slider is keyboard-accessible.

All user-facing copy uses UK English spellings. The platform does not currently support internationalisation (i18n) beyond UK English.

---

20. Monitoring and Observability

The following events are written to the audit log:

- Session start, pause, resume, completion, expiry
- Gaming pattern detection (pattern name, threshold vs observed count)
- Failure mode detection (mode name, item ID)
- LLM quality gate failures (checker name, item ID)
- Circuit breaker state changes
- Scoring config cache invalidation

Session metadata captures the following at session start and completion:

- `scoringConfigVersionAtStart` / `scoringConfigVersionAtCompletion`
- `normGroupVersionAtStart` / `normGroupVersionAtCompletion`
- `antiGamingOutcomeConditionalActive`
- `phaseOrderVariant`
- `resumeCount`
- `pinnedModelVersion`

---

21. Rollback Procedures

21.1 Scoring Config Rollback

The `scoring_config` table maintains both the v2.1 and v2.2 rows. To roll back to v2.1 scoring:

1. Set `is_active = false` on the v2.2 row.
2. Set `is_active = true` on the v2.1 row.
3. Call `invalidateScoringConfigCache()` to flush the 5-minute cache.

In-flight sessions pinned to v2.2 will continue to use v2.2 scoring until they complete. New sessions will use v2.1.

21.2 Feature Flag Rollback

To disable the outcome-conditional anti-gaming detector:

```
ANTI_GAMING_OUTCOME_CONDITIONAL=false
```

Restart the server to apply. Sessions already in progress will not be affected until the next answer submission.

21.3 Platform Checkpoint Rollback

The platform uses versioned checkpoints. To roll back to the v2.1 checkpoint (ea81412f):

1. Open the Management UI.
2. Navigate to Version History.
3. Select checkpoint `ea81412f` and click Rollback.

---

22. Glossary

Term	Definition
Signal delta	An unsigned magnitude (0.0–3.0) representing the strength of a signal observation from a single answer option
Outcome class	The qualitative classification of an answer option: `strong`, `acceptable`, `weak`, `failure`, or `critical_failure`
Capability domain	One of six high-level competency areas: execution, judgement, governance, appropriateness, workflow, data interpretation
Failure mode	A systematic pattern of governance or safety failures detected across multiple answers
Gaming pattern	A behavioural pattern suggesting the participant is responding strategically rather than authentically
Trap item	An item injected by the anti-gaming engine to probe a suspected gaming pattern
Readiness state	The overall classification: `safe` (AI-Ready), `at_risk` (Developing), `unsafe` (Not Yet Ready), or `unknown`
Confidence profile	A composite score (0–1) measuring the reliability of the classification based on evidence depth, breadth, diversity, and consistency
Norm group	A reference population against which capability scores are percentile-ranked
AIL	Adaptive Intelligence Layer — the post-assessment pipeline that produces user intelligence profiles and learning plans
Sum+clip formula	The v2.2 scoring formula: `clip(intercept + clip(sum, -cap, +cap) × multiplier, 0, 100)`
Circuit breaker	A safety mechanism that disables LLM generation after 3 consecutive failures, routing to the fallback item bank

---

23. Changelog — v2.2

This section records all changes introduced in the v2.2 Engine Remediation Work Package. Each change is linked to its work stream identifier.

WS1.1 — Sum+Clip Scoring Formula

Problem: The v2.1 mean-based formula violated monotonicity. Adding a positive-delta answer to a session with many existing answers could lower the capability score because the scale factor decreased faster than the new delta raised the average. This produced counter-intuitive results and eroded participant trust.

Change: Replaced the mean-based formula with a sum-and-clip formula in `computeCapabilityScores`. The new formula clips the signal sum at ±8.0 and multiplies by 6.25 before adding the 50-point intercept. The v2.1 formula is preserved and activated for sessions pinned to a v2.1 scoring config row.

Calibration: The `contributionCap=8.0` and `contributionMultiplier=6.25` defaults were calibrated against four synthetic anchor cases (A/B/C/D) to ensure the formula produces scores in the expected ranges for all-strong, all-failure, mixed, and monotonicity test sequences.

Test coverage: `scoring.v2-2.test.ts` — 8 tests covering all four anchor cases, boundary clamping, legacy fallback, and output completeness.

WS1.2 — Configurable Failure Mode Thresholds

Problem: The v2.1 failure mode detector used hard-coded thresholds. A single blind-acceptance answer could trigger a blocking classification, which was too sensitive for real-world use.

Change: Added `blockingFailureMinItems` (default: 2) and `downgradeFailureMinItems` (default: 1) to the scoring config. The `detectFailureModes` function now accepts these thresholds as parameters and applies them when counting unique failure modes. The default of 2 distinct blocking modes for a block classification matches the v2.1 intent but is now configurable per deployment.

Test coverage: `failure-modes.v2-2.test.ts` — 13 tests covering default thresholds, configurable thresholds, event code triggers, and non-blocking modes.

WS2.1 — Outcome-Conditional Anti-Gaming Detection

Problem: The anti-gaming engine did not detect the outcome-conditional gaming strategy, where participants give safe answers on high-risk items and strong answers on low-risk items to maximise their score while minimising governance failure risk.

Change: Added the `outcome_conditional_safe` pattern to `analyseGamingPatterns`. The pattern fires when >70% of high-risk answers are `acceptable` and >70% of low-risk answers are `strong`, with a minimum of 8 total answers and 3 answers in each risk tier. When detected, a governance-domain trap item is injected. The feature is controlled by `ANTI_GAMING_OUTCOME_CONDITIONAL` (default: `true`).

Test coverage: `anti-gaming.outcome-conditional.test.ts` — 16 tests covering pattern detection, minimum answer thresholds, feature flag, role-aware thresholds, and audit event codes.

WS2.2 — Role-Aware Anti-Gaming Thresholds

Problem: The anti-gaming engine applied the same detection thresholds regardless of the participant's role or seniority. This produced false positives for specialist roles (where always-safe behaviour is appropriate) and missed gaming by senior users who should demonstrate stronger performance.

Change: Added `ROLE_GAMING_THRESHOLDS` with specialist and leader overrides. Added the `seniority_inconsistent` pattern, which fires when a senior user (seniority ≥ 3) has >50% weak answers. The pattern is suppressed for junior users.

Test coverage: Covered in `anti-gaming.outcome-conditional.test.ts`.

WS2.3 — Extended Audit Event Codes

Problem: The `GAMING_AUDIT_EVENT_CODES` map was incomplete, missing codes for `outcome_conditional_safe` and `seniority_inconsistent`.

Change: Added codes for all 11 `GamingPattern` types to `GAMING_AUDIT_EVENT_CODES`.

Test coverage: Covered in `anti-gaming.outcome-conditional.test.ts`.

WS3 — LLM Item Quality Gate Enhancements

Problem: The LLM quality gate lacked a PII sanitiser, vendor allow-list, and bias detector. Items containing email addresses, specific AI product names, or gendered language could be served to participants.

Change: Added three new checkers to `runQualityGate`:

- `pii_sanitiser` — strips email addresses, UK phone numbers, and NI numbers
- `vendor_allow_list` — rejects items referencing ChatGPT, Claude, Copilot, Gemini, Bard, or GPT-4

- `bias_detector` — rejects items containing gendered pronouns, age-related language, or disability-related language

The circuit breaker was formalised: 3 consecutive failures within 60 seconds opens the gate. `buildReviewQueueRow` was added to produce a structured DB row for the review queue.

Test coverage: `llm-checkers.test.ts` — 27 tests covering all 4 checkers, circuit breaker, and review queue row construction.

WS4.2 — Classification Explanation Procedure

Problem: Participants received a readiness classification with no explanation of the factors that drove it.

Change: Added the `getClassificationExplanation` tRPC procedure. It returns a structured explanation including `topStrengths`, `topGaps`, a `factors` array with direction and detail for each contributing factor, `isProvisional`, and `confidenceBand`. The procedure enforces ownership: only the session owner can retrieve the explanation.

Test coverage: `classification-explanation.test.ts` — 22 tests covering all logic branches.

WS4.4 — Save-and-Resume 48-Hour Window

Problem: The save-and-resume feature had no formal time limit. Sessions could be resumed indefinitely, creating scoring inconsistencies when the item bank or scoring model changed between pause and resume.

Change: The `getResumeState` procedure now computes `isResumeWindowOpen` (true if < 48 hours since last activity), `resumeWindowExpiresAt` (exact expiry timestamp), and `progressPct` (percentage of target items answered). Sessions outside the window are marked `expired`.

Test coverage: `save-resume.test.ts` — 19 tests covering the 48h window, edge cases (exactly 48h, 47h, 72h), expiry timestamp computation, and progress percentage rounding.

WS4.5 — Model Version Pinning

Problem: When a session was resumed after a model update, the adaptive engine used the new model version, potentially generating items with different characteristics than those answered before the pause.

Change: The LLM model version is pinned to session metadata as `pinnedModelVersion` when the session is paused. `getResumeState` returns the pinned version. The adaptive engine uses the pinned version on resume. Default: `"adaptive-v2"`.

Test coverage: Covered in `save-resume.test.ts`.

WS1.2 (Correction) — Item-Counting Semantics for Failure Mode Thresholds

Problem: The initial v2.2 implementation of `detectFailureModes` counted unique failure mode types against the `blockingFailureMinItems` threshold. This was inconsistent with the intent of the threshold name ("MinItems") and produced unexpected results when a single mode appeared many times.

Change: Corrected `detectFailureModes` to count the total number of answer-level failure mode occurrences (blockCount) against the threshold, not the number of unique mode types. The architecture document Section 5.2 was updated to reflect this semantics. Two regression tests were added to `failure-modes.v2-2.test.ts`.

Rationale: Item-counting is more robust: it requires a pattern to appear multiple times before triggering a block, which is the intended behaviour for a "minimum items" threshold.

WS1.3 — Contribution Breakdown JSON

Problem: The `assessment_answers` table lacked a structured breakdown of which signals each answer contributed to, making it impossible to trace score movements to specific answers in the back-office.

Change: Added `contribution_breakdown_json` column to `assessment_answers` (migration 0018). The `submitAnswer` procedure now populates this column with a JSON object mapping capability domain keys to their signal delta contribution from that answer. A JSON path index was added for efficient back-office queries.

WS2.2 — Anti-Gaming Thresholds Database Table

Problem: Anti-gaming thresholds were hard-coded in `antiGamingEngine.ts`. There was no way to adjust thresholds per deployment without a code change.

Change: Added `anti_gaming_thresholds` table (migration 0018) with columns for role tier, pattern type, always-safe rate threshold, strong-answer rate threshold, minimum answer count, and audit metadata. Added `backoffice.getAntiGamingThresholds` and `backoffice.upsertAntiGamingThreshold` tRPC procedures. Added a back-office admin tab for managing thresholds.

WS3 — LLM Review Queue Back-Office UI

Problem: The `llm_item_review_queue` table was populated by the quality gate but had no back-office UI for reviewing flagged items.

Change: Added `backoffice.getLlmReviewQueue` tRPC procedure and a back-office tab that lists flagged items with their checker, severity, and review status. Admins can mark items as reviewed.

WS4.1 — Persona Label Softening

Problem: Participant-facing persona labels used clinical internal terminology (e.g., "Overconfident Decision-Maker", "Passive Deferrer") that could be perceived as judgemental.

Change: Added `PERSONA_SOFTENED_LABELS` map and `getPersonaLabel()` helper to `featureFlags.ts`. Added `PERSONA_LABEL_SOFTENING_ENABLED` feature flag (default: `true`). Applied `getPersonaLabel()` in `capabilityReport.ts` and `personaClassificationEngine.ts` (narrative generation). When enabled, labels are replaced with participant-appropriate alternatives (e.g., "Decisive Practitioner", "Collaborative Deferrer").

WS4.3 — Session Review Flags Back-Office UI

Problem: The `assessment_review_flags` table was populated by participants but had no back-office UI for reviewing flagged sessions.

Change: Added `backoffice.getReviewFlags` tRPC procedure and a back-office tab that lists flagged sessions with their reason, timestamp, and session ID.

WS5.1 — Telemetry Column Additions

Problem: The `assessment_answer_telemetry` table was missing several columns specified in the WS5.1 telemetry spec: `time_to_first_interaction_ms`, `confidence_rating_raw`, `answer_revision_count`, `focus_loss_count`, and `scroll_depth_pct`.

Change: Added all five columns to `assessment_answer_telemetry` (migration 0018). The `submitAnswer` procedure now accepts and writes all five fields. The `AssessmentSessionPage` frontend now captures `timeToFirstInteractionMs` (time from item display to first option click) and passes it with each answer submission.

WS5.2 — Session Metadata Additions

Problem: The `assessment_sessions` table was missing session-level metadata fields needed for cohort analysis: `device_type`, `locale_code`, `norm_group_version`, and

`scoring_config_version_at_start`.

Change: Added all four columns to `assessment_sessions` (migration 0018). The `startSession` procedure now accepts `deviceType` and `localeCode` from the frontend and writes `scoringConfigVersionAtStart` from the active scoring config at session creation time. `normGroupVersion` defaults to `"v1"` and is configurable via the scoring config.

Cross-Cutting — Feature Flag Additions

Problem: The `LLM_CHECKER_ENABLED`, `SAVE_AND_RESUME_ENABLED`, and `PERSONA_LABEL_SOFTENING_ENABLED` flags were not implemented, making it impossible to disable these features in CI or development environments without code changes.

Change: Added all three flags to `featureFlags.ts` with documented defaults. Wired `LLM_CHECKER_ENABLED` into the `submitAnswer` procedure (bypasses the quality gate when disabled). Wired `SAVE_AND_RESUME_ENABLED` into `getResumeState` (returns `{ isResumeWindowOpen: false, reason: 'feature_disabled' }` when disabled).

Cross-Cutting — Signal Inventory Correction (Section 3.1)

Problem: Section 3.1 of this document listed 17 canonical signals. The `SIGNAL_TO_CAPABILITY` map in `scoringEngine.ts` contains 22 signals.

Change: Section 3.1 updated to list all 22 signals with correct capability mappings. A full three-source audit (code, DB migration 0009, architecture doc) was completed in the Claude Feedback Round 3 pass and confirmed all 22 signals are in agreement across all sources. See `docs/signal-mapping-audit-v2.2.md` for the full comparison table.

Claude Feedback Round 3 — Addition A: E3 Hybrid Failure-Mode Semantics

Problem: The v2.2 Completion Pass reverted E3 (unique-mode counting, introduced in the Adaptivity Review commit `10350c0`) to item-counting. However, E3 addressed a valid concern: a participant who triggers the same blocking mode 10 times should not accumulate `blockCount = 10`. Pure item-counting is disproportionate for repeated single-pattern failures.

Change: Implemented a hybrid: `classificationImpact = 'block'` requires **both** `itemCount >= 2` **and** either `distinctModeCount >= 2` **or** `itemCount >= baseThreshold * 1.5`. This preserves E3's protection against single-pattern inflation while ensuring that genuine multi-mode failures (e.g. 5 different blocking modes, each triggered once) still block. The `blockCount` field in `FailureModeResult` is retained for back-office audit purposes. Tests updated in `server/failure-modes.v2-2.test.ts` with 4 new hybrid-specific cases.

Claude Feedback Round 3 — Addition B: Provisional Confidence Band [0.40, 0.50)

Problem: The `classifyReadiness` function used a single `CONFIDENCE_FLOOR = 0.50` threshold: any composite confidence below 0.50 returned `unknown_insufficient_evidence`. The architecture document described a two-threshold model (0.40 provisional, 0.50 insufficient evidence) but the code only implemented one threshold.

Change: Added `PROVISIONAL_CONFIDENCE_THRESHOLD = 0.40` constant and `isProvisional: boolean` field to `ReadinessClassification`. The confidence floor check now fires at `< 0.40` (returning `unknown_insufficient_evidence`). Values in `[0.40, 0.50)` fall through to normal classification with `isProvisional = true`. All four classification return paths (`unsafe`, `at_risk`, `safe`, default `at_risk`) now include the `isProvisional` flag. 20 regression tests added in `server/confidence-floor.test.ts`.

Three-threshold model (updated):

Threshold	Constant	Effect
`0.40`	`PROVISIONAL_CONFIDENCE_THRESHOLD`	Below this: `unknown_insufficient_evidence`
`[0.40, 0.50)`	—	Normal classification + `isProvisional = true`
`0.50`	`CONFIDENCE_FLOOR`	Retained as documentation; no longer a code gate
`0.55`	`MINIMUM_SAFE_CLASSIFICATION_CONFIDENCE`	Below this: `safe` → `at_risk` downgrade

Claude Feedback Round 3 — Addition C: 22-Signal Mapping Audit

Problem: Claude's feedback requested a formal audit confirming that the 22 canonical signals are consistent across code, DB, and documentation.

Change: Full three-source audit completed. All 22 signals confirmed in agreement across: (1) `SIGNAL_TO_CAPABILITY` in `scoringEngine.ts`, (2) `INSERT IGNORE` rows in `drizzle/0009_signal_key_constraints.sql`, and (3) Section 3.1 of this document. Audit report written to `docs/signal-mapping-audit-v2.2.md`. No code changes required.

WS1.2 Item 1 — Six Hard-Coded Scoring Constants Made Configurable

Problem: The E3 hybrid delta threshold (`1.5`), catastrophic margin multiplier (`1.5`), confidence floor (`0.50`), provisional confidence threshold (`0.40`), at-risk confidence floor (`0.35`), and

minimum-safe classification confidence (`0.55`) were compile-time constants in `scoringEngine.ts` and `classificationConfidenceGate.ts`. They could not be adjusted without a code change and redeploy.

Change: All six constants are now columns in `scoring_config` (migration `0019_configurable_scoring_thresholds.sql`). `getActiveScoringConfig()` reads and caches them. They are threaded through the call chain: `assessment.ts` router → `SessionController.computeResults` → `detectFailureModes` (base threshold + catastrophic margin) → `classifyReadiness` (provisional threshold + confidence floor) → `applyClassificationConfidenceGate` (safe threshold + at-risk floor). All defaults reproduce the former hard-coded behaviour exactly — no behaviour change for existing sessions.

Migration: `0019_configurable_scoring_thresholds.sql` — adds six `DECIMAL(5,3)` columns with `NOT NULL DEFAULT` values matching the former constants. The existing v2.2 row (version=2) is updated to carry these values explicitly.

Test coverage: `server/scoring-config-overrides.test.ts` — 22 tests covering: `baseFailureThresholdMagnitude` (raised/lowered), `catastrophicMarginMultiplier` (lowered to trigger catastrophic path), `atRiskConfidenceFloor` (raised), `minimumSafeClassificationConfidence` (lowered/raised), `provisionalConfidenceThreshold` (raised), `confidenceFloor` (raised), and default-identity (explicit defaults produce identical results to no opts).

WS1.2 Item 2 — `isProvisional` Semantics Clarified (Option A)

Problem: The `isProvisional` flag had ambiguous semantics. The question was whether it should fire when a `safe` classification is downgraded to `at_risk` by the confidence gate (Option B), or only when composite confidence is in the provisional band `[0.40, 0.50)` (Option A).

Decision: Option A. `isProvisional` has narrow, band-specific semantics: it fires only when `compositeConfidence ∈ [provisionalConfidenceThreshold, confidenceFloor)`. The gate-downgrade case (safe → at_risk at confidence < 0.55) is already surfaced via the caveat banner on the headline result card (`applyClassificationConfidenceGate` returns `wasDowngraded=true` and a `caveat` string). Adding `isProvisional=true` to the gate-downgrade case would create a second, different signal on the same amber banner, confusing participants.

Code change: The `isProvisional` computation in `classifyReadiness` now uses the configurable `provThreshold` and `confFloor` variables (introduced by Item 1) instead of the module-level constants. The code comment is updated to document the narrow semantics explicitly.

End of document.